

Sub-filter energy identity — Lean 4 / Mathlib formalisation

Report for TASK_BRIEF.md. Everything in Parts 1, 2 and 3 compiles with **no sorry**, and every theorem depends only on propext, Classical.choice, Quot.sound.

Subfilter/Identity.lean	Part 1: Steps 1–4, identity (1), Leonard decomposition
Subfilter/Model.lean	Part 2: two-member polynomial ensemble, genuine Reynolds operator
Subfilter/Analytic.lean	Part 3: Step 5 (Doppler weight), Step 6 (integration by parts)
reference/	note (tex/pdf) and the sympy check script, unchanged

1. Hypotheses actually used, theorem by theorem

The column "uses" was extracted mechanically from the proof terms (transitive constant closure inside the Subfilter namespace, tested for Derivation.leibniz, Setup.Av_comm, Setup.Av_res); the hypothesis column lists the explicit hypotheses of the signature, all of which are used (Lean's unused-variable linter is on and silent). This table can replace §9 of the note.

theorem	informal statement	Leibniz	Av_comm	Av_res	idempotence	div u = 0	s resolved	other
dot_cross_cyclic (A1)	$a \cdot (b \times c) = b \cdot (c \times a)$							commutative ring only
dot_cross_swap (A2)	$a \cdot (b \times c) = -a \cdot (c \times b)$							commutative ring only
step1	$b \times \text{curl } b = \text{gradHalfSq } b - (b \cdot \nabla) b$							ε - δ identity only (derivations enter as symbols, Leibniz never used)
d_sq, gradHalfSq_eq, two_smul_gradHalfSq (1')	$\partial(\text{ff}) = 2f\partial f$; $\text{gradHalfSq } b = \frac{1}{2}\nabla(b \cdot b)$	✓						
step2	$(u \cdot \nabla) u = \text{divT}(u \otimes u)$	✓				✓		
pointwise (P)	$u \cdot (s \times \omega) = s \cdot \text{divT}(u \otimes u) - s \cdot \text{gradHalfSq } u$	✓				✓		
Av_dot_res (3a)	$\text{Av}(s \cdot X) = s \cdot \text{Av } V \cdot X$			✓			✓	linearity of Av
AvV_divT (3b)	$\text{Av } V(\text{divT } T) = \text{divT}(\text{Av } T)$		✓					linearity of Av
AvV_gradHalfSq (3c)	$\text{Av } V(\text{gradHalfSq } u) = \frac{1}{2}\nabla \text{Av}(u \cdot u)$	✓	✓					linearity of Av
identity_one (M)	(1)	✓	✓	✓	no	✓	✓	no $\text{Av } u = 0$, no commuting derivatives
identity_one_two_smul	(1) with the $\frac{1}{2}$ cleared	✓	✓	✓	no	✓	✓	corollary of (M)
sym_antisym (4a)	R symmetric $\Rightarrow \sum R_{ij}(\partial_i s_j - \partial_j s_i) = 0$							commutative ring only
div_flux, dot_divT_outer	expansions used by 4b	✓				✓		
step4_split (4b)	$u \cdot (s \times \omega) = \text{div}(u(s \cdot u)) - s \cdot \text{gradHalfSq } u - \sum u_i u_j \partial_j s_i$	✓				✓		4a with $R_{ij} = u_i u_j$

theorem	informal statement	Leibniz	Av_comm	Av_res	idempotence	div u = 0	s resolved	other
step4_split_av (eq. s4 of the note, averaged)	$Av(u \cdot (s \times \omega)) = \text{div } AvV(u(s \cdot u)) - s \cdot \nabla k_t - \sum Av(u_i u_j) \partial_j s_i$	✓	✓	✓	no	✓	✓	plus $\partial_j s_i$ resolved (see §2)
leonard (L1)	$Av(u_i u_j) - U_i U_j = L + C + Av(u'_i u'_j)$							linearity of an arbitrary Av only
leonard_vanish (L2)	Reynolds axioms $\Rightarrow L = 0, C = 0$			✓	✓			Av u resolved

Acceptance criteria of the brief:

- take build: success, zero sorry (grep of the build log and of the sources: 0 hits).
- #print axioms for identity_one, identity_one_two_smul, step4_split, step4_split_av, leonard, leonard_vanish, Model.setup, Model.identity_one_model, Analytic.*: [propext, Classical.choice, Quot.sound] in every case.
- No statement was weakened. identity_one assumes exactly $\text{div } u = 0$ and $\forall i, s_i \in \text{resolved}$; none of $Av \ u = 0$, idempotence, or $d \ i \circ d \ j = d \ j \circ d \ i$ is needed anywhere in Steps 1–4. The note's derivation is confirmed on this point.
- Every theorem carries a docstring naming its Step and the axioms it uses.
- example : Setup (Fin 2 → MvPolynomial (Fin 3) ℝ) := Model.setup is in Model.lean.

2. Statements corrected or added

- **None of the 13 listed statements had to be changed.**
- **One addition with an extra hypothesis.** The *averaged* three-term split, eq. (s4) of the note (step4_split_av, not in the brief's list), needs $\partial_j s_i \in \text{resolved}$ in addition to $s_i \in \text{resolved}$: the Stokes-production term $Av(u_i u_j \partial_j s_i)$ can be written as $Av(u_i u_j) \partial_j s_i$ only if the *derivatives* of s pass through the average. The subalgebra resolved is not assumed closed under $d \ i$ (and in the model it is closed, but the axioms do not say so). The identity (1) itself does not need this, because in (1) the derivatives fall on u and on $Av(u \cdot u)$, never on s. For §9 item 2 of the note: " u^s passes through the average" should read " u^s **and its gradient** pass through the average" whenever the Stokes-production reading (s4) is used; for (1) alone the gradient condition is not needed.
- Part 3, Step 6 is stated twice: step6_parts with HasDerivAt hypotheses and interval-integrable derivatives (the general Mathlib form), and step6_parts_deriv in the brief's wording (differentiable on $[-h, 0]$, deriv continuous there). Step 5 includes the optional normalisation $\int_{\{Iic \ 0\}} 2k e^{\{2kz\}} dz = 1$ (integral_exp_mul_Iic).

3. Part 2: which model

The full suggested model, not the fallback: $R := \text{Fin } 2 \rightarrow \text{MvPolynomial } (\text{Fin } 3) \ \mathbb{R}$, $d \ i = \text{componentwise MvPolynomial.pderiv } i$ (a Derivation $\mathbb{R} \ R \ R$ built by hand), $Av \ f = \text{fun } _ \Rightarrow \frac{1}{2} (f \ 0 + f \ 1)$, $\text{resolved} = \{f \mid f \ 0 = f \ 1\}$ as a Subalgebra $\mathbb{R} \ R$. Proved: Av_comm , Av_res (the axioms), Av_idem , $Av_fluct : Av \ (f - Av \ f) = 0$, $Av_mem_resolved$; non-degeneracy: $\text{resolved} \neq \top$, $Av \neq \text{id}$, $\text{piDeriv } i \neq 0$; and identity_one_model, the identity (1) instantiated on a member-dependent solenoidal $u = (\pm X_1, 0, 0)$ with $Av \ u = 0$ and a non-constant resolved $s = (X_2, 0, 0)$.

4. What in the skeleton's intended proofs was wrong

Mathematically nothing. Lean-level fixes:

1. A module docstring cannot precede import; imports moved to the top.
2. $S.\text{curl}$, $S.\text{div}$, ... (dot notation on $S : \text{Setup } R$) only resolve if the operators live in the Setup namespace; the definitions were moved into namespace Setup (theorem names unchanged: Subfilter.identity_one etc.).

3. `import Mathlib` was replaced by targeted imports (see §5). `Mathlib.Data.Real.Basic` is deprecated since 2026-08-27; $\mathbb{R}$ now comes from `Mathlib.Basic.Real.Basic`.
4. The $(2:\mathbb{R})^{-1} \cdot \text{juggling in gradHalfSq_eq}$ was isolated in `half_smul_two_mul : (2:\mathbb{R})^{-1} \cdot (2 * x) = x`; the skeleton's inline `show ... by rw [two_mul, two_smul]` was not needed.
5. `step4_split` went through exactly as planned (Leibniz on `d i (u i (s·u))`), `div u = 0`, compare with `(P)`, cancel with `sym_antisym` on `R_ij = u_i u_j`; `linear_combination` was avoided to keep the import set small (`sub_eq_zero.mp + ring` suffices).
6. `dot_cross_cyclic`, `dot_cross_swap`, `step1`, `pointwise`, `identity_one`, `leonard`, `leonard_vanish` compiled as written, modulo the two points above.
7. In the model, `field_simp/simp` closed some goals outright, so trailing rings were removed; `MvPolynomial` is noncomputable, so `Model.lean` is a noncomputable section.

5. Environment, versions, build

- Lean `leanprover/lean4:v4.35.0-rc4`; Mathlib master at `9e6b3aac99b6` (see `lake-manifest.json`).
- The environment's network policy blocks `release.lean-lang.org`, `reservoir.lean-lang.org` and `cache.mathlib.org` (403 on CONNECT). The toolchain was therefore installed from the GitHub release tarball, Mathlib was required by git URL instead of Reservoir, and — against the brief's advice, there being no alternative — the needed part of Mathlib was **built from source**: 2 850 Mathlib modules (about 1 400 for Parts 1–2, the rest for the measure theory of Part 3), about 60 minutes on 4 cores (25 min for the Parts 1–2 closure, 35 min for the Part 3 closure). With cache access, `lake exe cache get` replaces that step.
- Remaining build warnings are only Mathlib's note that its files are "designed for use with the module system; consider adding module"; the project uses classic (non-module) files on purpose so that `#print axioms` and dot-notation behave as in the brief.

Exact tail of the final lake build:

```

△ [2863/2867] Replayed Subfilter.Analytic
△ [2864/2867] Built Subfilter.Identity (3.7s)
△ [2865/2867] Built Subfilter.Model (2.7s)
✓ [2866/2867] Built Subfilter (2.8s)
Build completed successfully (2867 jobs).
```

(the `△` are the module-system notes above; no sorry, no error.)

Axiom check (`lake env lean` on a file importing `Subfilter`):

```

'Subfilter.identity_one' depends on axioms: [propext, Classical.choice, Quot.sound]
'Subfilter.identity_one_two_smul' depends on axioms: [propext, Classical.choice, Quot.sound]
'Subfilter.step4_split' depends on axioms: [propext, Classical.choice, Quot.sound]
'Subfilter.step4_split_av' depends on axioms: [propext, Classical.choice, Quot.sound]
'Subfilter.leonard' depends on axioms: [propext, Classical.choice, Quot.sound]
'Subfilter.leonard_vanish' depends on axioms: [propext, Classical.choice, Quot.sound]
'Subfilter.Model.setup' depends on axioms: [propext, Classical.choice, Quot.sound]
'Subfilter.Model.identity_one_model' depends on axioms: [propext, Classical.choice, Quot.sound]
'Subfilter.Model.Av_idem' depends on axioms: [propext, Classical.choice, Quot.sound]
'Subfilter.Model.resolved_ne_top' depends on axioms: [propext, Classical.choice, Quot.sound]
'Subfilter.Analytic.step5_weight' depends on axioms: [propext, Classical.choice, Quot.sound]
'Subfilter.Analytic.step5_weight_integral' depends on axioms: [propext, Classical.choice, Quot.sound]
'Subfilter.Analytic.step6_parts' depends on axioms: [propext, Classical.choice, Quot.sound]
'Subfilter.Analytic.step6_parts_deriv' depends on axioms: [propext, Classical.choice, Quot.sound]
```

6. Reproducing

```

cd subfilter
lake exe cache get      # if cache.mathlib.org is reachable; otherwise lake build compiles ~2850
```

```
Mathlib files
lake build
```